



Projet MyBank – Documentation

1. Conception

1.1. Zoning

- **Page Login** : formulaire (email + mot de passe), bouton “Login”, lien “Sign up”.
- **Dashboard** : aperçu rapide du solde et des dernières opérations.
- **Liste des dépenses** : tableau avec libellé, montant, date, catégorie.
- **Ajout/Mise à jour dépense** : formulaire (libellé, montant, date, catégorie).
- **Catégories** : liste + bouton d’ajout/modification.

1.2. Wireframe (description textuelle)

- **Header** : logo + menu de navigation (Dashboard, Expenses, Categories, Logout).
- **Sidebar (optionnel)** : filtres (par catégorie, par date).
- **Content** : tableau ou formulaire selon la section.

1.3. Maquette (principes graphiques)

- **Typo** : Montserrat.
 - **Couleurs** : #59e5a9 (primaire), #000000 (texte), #14213d (fond sombre), #fca311 (accent), #ec0b43 (erreurs).
 - **Style** : minimaliste, clair, responsive.
-

2. Journal de développement

Jour 1 : Initialisation

- Création dépôt GitHub mybank.
- Configuration Docker (containers pour backend Symfony + frontend React + DB).

Jour 2 : Backend

- Création API Symfony : CRUD **expenses** et **categories**.
- Tests Postman pour valider endpoints.

Jour 3 : Frontend

- Setup React (Vite).
- Création pages Login + Dashboard.
- Intégration API (fetch expenses).

Jour 4 : CI/CD

- Ajout GitHub Actions workflow (test + build + docker push).
- Rédaction premiers tests d'intégration.

Jour 5 : Finalisation

- Amélioration UI (style + responsive).
 - Documentation CI/CD.
 - Préparation archive finale.
-

3. Recherches & veille technique

- **Docker** : utilisation de docker-compose.yml pour orchestrer backend + frontend + DB.
 - **Symfony API** : choix de API Platform pour rapidité.
 - **React** : Hooks (useState, useEffect) pour gérer données.
 - **GitHub Actions** : automatisation build/test/deploy.
 - **Tests** : PHPUnit pour backend, Jest pour frontend.
-

4. Documentation CI/CD

4.1. Installation Docker

- **Windows/macOS** : installer Docker Desktop.
- **Linux** : `sudo apt install docker docker-compose`.

4.2. Pourquoi Docker ?

- Même environnement pour dev, test, prod.
- Isolation et portabilité.

4.3. CI/CD Pipeline (GitHub Actions)

name: CI/CD MyBank

on:

push:

branches: [main]

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3
- name: Install dependencies
run: npm install --prefix frontend
- name: Run tests

```
    run: npm test --prefix frontend
- name: Build Docker images
  run: docker-compose build
- name: Push Docker images
  run: docker-compose push
- name: Deploy
  run: ssh user@server 'cd mybank && docker-compose pull && docker-
compose up -d'
```

4.4. Déploiement continu

- Chaque push sur main : build + test + push image Docker + déploiement sur serveur.
- Serveur met à jour containers automatiquement.

4.5. Tests d'intégration

- Vérifier qu'une dépense ajoutée via frontend est bien reçue par backend et stockée DB.
- Vérifier suppression/modification.

5. README.md

MyBank

MyBank est une application web permettant de gérer ses dépenses facilement.

🚀 Stack

- Frontend : React
- Backend : Symfony (API)
- Base de données : PostgreSQL
- Conteneurisation : Docker
- CI/CD : GitHub Actions

🔗 Installation

```
``bash
git clone https://github.com/username/mybank.git
cd mybank
docker-compose up --build
```

🔍 Tests

- Backend : php bin/phpunit
- Frontend : npm test

📦 Déploiement

Automatisé via GitHub Actions + Docker.



Auteur

Samir NZAMBA – CDA04 ``

6. Procédure de déploiement

1. Cloner repo GitHub.
2. Installer Docker & Docker Compose.
3. Lancer `docker-compose up -d --build`.
4. Vérifier accès via navigateur (<http://localhost:3000>).
5. CI/CD se charge ensuite de maintenir le projet en ligne.